

Drug Path Visualiser

Professional MVP Implementation Roadmap

Project Overview

Drug Path Visualiser is a mobile-friendly educational web application that helps general users understand what happens to an orally taken drug through an interactive 3D abstract pathway. The MVP focuses on Ibuprofen, with Before Food / After Food as the only dynamic pathway condition.

Project Goals

- Build a clear educational visualization of the Ibuprofen pathway.
- Keep scientific content separate from 3D presentation logic.
- Create an architecture that can support additional drugs later.
- Provide anonymous access with optional Google sign-in.
- Implement secure backend validation and authorization.
- Build a maintainable modular-monolith architecture.
- Establish a strong testing and integration process.

Technology Stack

- Frontend: React + TypeScript
- 3D: Three.js + React Three Fiber
- Backend: Node.js + TypeScript
- Architecture: Modular Monolith
- Database: PostgreSQL
- API: REST over HTTPS
- Authentication: Google OAuth/OIDC
- Testing: Unit, API, integration, security and E2E testing
- Version Control: Git

Architecture

Browser → React + TypeScript + Three.js/R3F → REST/HTTPS → Node.js + TypeScript modular monolith → PostgreSQL.
Backend modules: Authentication, Drug & Pathway, Content, Quiz, Progress and Administration.

Total Tasks: 30

Estimated Duration: Approximately 6–8 weeks for a 3-person team.

Difficulty: Intermediate → Advanced

Team Structure

Bucket	Owner	Primary Responsibility
Tasks 01–10	Member 1	Frontend, UI & 3D
Tasks 11–20	Member 2	Backend, Database & APIs
Tasks 21–30	Member 3	Content, Quiz, Security, Testing & Integration

■ Task 01 — Project Repository and Development Setup

Create the initial project repository and establish common development conventions so frontend, backend and testing work can be integrated without conflicting assumptions.

CONCEPTS

- Git repository
- Branching strategy
- Environment configuration
- Project documentation

ACCEPTANCE CRITERIA

- Repository is created with main and develop branches.
- .gitignore prevents secrets, dependencies and generated files from being committed.
- .env.example defines required variables without real secrets.
- README contains setup and run instructions.

FILES / FOLDERS

README.md
.gitignore
.env.example
docs/
package.json

DELIVERABLES

- Initial repository
- Development setup documentation
- Git workflow

DEPENDENCIES

Task 01 has no dependencies.

ESTIMATED TIME

2–3 Hours

NOTES

Keep the repository simple. ■ Do not commit real API keys, OAuth secrets, database passwords or production credentials.

■ Task 02 — Frontend Application Bootstrap

Create the React + TypeScript frontend and establish the basic frontend structure. The application should run independently before real API integration.

CONCEPTS

- React
- TypeScript
- Component architecture

- Development server

ACCEPTANCE CRITERIA

- React application starts successfully.
- TypeScript compilation succeeds.
- Development server runs consistently.
- Basic component structure is established.

FILES / FOLDERS

```
frontend/  
src/main.tsx  
src/App.tsx  
src/components/  
src/pages/  
package.json  
tsconfig.json
```

DELIVERABLES

- Working React application
- Initial component structure
- TypeScript configuration

DEPENDENCIES

Task 01.

ESTIMATED TIME

2–3 Hours

NOTES

Use reusable components rather than putting the application inside App.tsx.

■ Task 03 — Frontend Routing and Application Shell

Create the basic page structure and navigation. The shell provides a stable foundation for explorer, quiz and authentication screens.

CONCEPTS

- Client-side routing
- Layout components
- Navigation
- Page composition

ACCEPTANCE CRITERIA

- Main routes are defined.

- Shared layout is reusable.
- Navigation works without full page reloads.
- Unknown routes show a fallback page.

FILES / FOLDERS

```
src/App.tsx
src/routes/
src/layouts/
src/pages/
src/components/navigation/
```

DELIVERABLES

- Application shell
- Route configuration
- Navigation components

DEPENDENCIES

Task 02.

ESTIMATED TIME

2–3 Hours

NOTES

Avoid putting business logic into routing components.

■ Task 04 — Design System and Reusable UI Components

Create reusable UI components required by the application. This establishes consistency before the 3D explorer and quiz are implemented.

CONCEPTS

- Component reuse
- UI consistency
- Accessibility
- Responsive design

ACCEPTANCE CRITERIA

- Buttons, cards, panels and selectors are reusable.
- Components support required states.
- Keyboard interaction works for interactive controls.
- Layout works on desktop and mobile widths.

FILES / FOLDERS

```
src/components/ui/
```

src/styles/
src/components/common/

DELIVERABLES

- Reusable UI components
- Base styling
- Responsive layout foundation

DEPENDENCIES

Task 03.

ESTIMATED TIME

4–5 Hours

NOTES

Prefer composition over duplicated UI code. ■ Do not build every page with custom one-off controls.

■ Task 05 — API Client and Frontend Data Layer

Create the frontend API client that communicates with the backend. The API client follows the agreed contract rather than assumptions about backend implementation.

CONCEPTS

- REST clients
- HTTP requests
- JSON
- API contracts

ACCEPTANCE CRITERIA

- Central API client exists.
- API errors are handled consistently.
- Base API URL comes from environment configuration.
- API response types are defined in TypeScript.

FILES / FOLDERS

src/api/
src/api/client.ts
src/api/types.ts
src/services/
.env.example

DELIVERABLES

- API client
- API response types

- Error-handling foundation

DEPENDENCIES

Task 01, Task 02.

ESTIMATED TIME

3–4 Hours

NOTES

The API contract must be agreed with Member 2. ■ Do not scatter raw fetch calls throughout UI components.

■ Task 06 — Pathway Explorer UI

Build the main pathway exploration interface around the future 3D visualization. Placeholder pathway data may be used while backend integration is completed.

CONCEPTS

- Application state
- Explorer layout
- Information panels
- Loading/error states

ACCEPTANCE CRITERIA

- Explorer page is accessible.
- Pathway area and information panel are separated.
- Selected-node state works with mock data.
- Loading and error states exist.

FILES / FOLDERS

src/pages/ExplorerPage.tsx
src/components/pathway/
src/components/node/
src/state/

DELIVERABLES

- Explorer page
- Node information panel
- Explorer state management

DEPENDENCIES

Task 04.

ESTIMATED TIME

4–5 Hours

NOTES

Keep scientific content outside presentation components.

■ Task 07 — Three.js / React Three Fiber Foundation

Integrate Three.js and React Three Fiber and create the basic abstract 3D scene. The visualization should remain educational rather than photorealistic.

CONCEPTS

- Three.js
- React Three Fiber
- 3D scene
- Camera and lighting

ACCEPTANCE CRITERIA

- 3D scene renders successfully.
- Camera controls work.
- Scene works inside explorer.
- Rendering remains usable on normal hardware.

FILES / FOLDERS

```
src/components/3d/  
src/components/3d/PathwayScene.tsx  
src/components/3d/Camera.tsx  
src/components/3d/lighting/
```

DELIVERABLES

- Working 3D scene
- Camera setup
- Rendering configuration

DEPENDENCIES

Task 06.

ESTIMATED TIME

5–6 Hours

NOTES

Start with simple geometry. ■ Do not spend MVP time creating photorealistic anatomy.

■ Task 08 — Interactive Pathway Graph

Connect pathway nodes and edges to the 3D scene. Visualization must be driven by pathway data rather than hardcoded scientific assumptions.

CONCEPTS

- Graph visualization
- Nodes
- Edges
- Selection
- Data-driven rendering

ACCEPTANCE CRITERIA

- Nodes are rendered from data.
- Edges connect appropriate nodes.
- Selecting a node updates state.
- Additional nodes can be accommodated without rewriting the scene.

FILES / FOLDERS

src/components/3d/PathwayScene.tsx
src/components/3d/PathwayNode.tsx
src/components/3d/PathwayEdge.tsx
src/types/pathway.ts

DELIVERABLES

- Data-driven pathway graph
- Node selection
- Edge rendering

DEPENDENCIES

Task 07.

ESTIMATED TIME

5–6 Hours

NOTES

- Do not create drug-specific components such as IbuprofenStomachNode.

■ Task 09 — Food Condition Interaction

Implement the Before Food / After Food control and connect it to pathway state according to the backend contract.

CONCEPTS

- State management
- Parameters
- Variations
- Conditional rendering
- API integration



ACCEPTANCE CRITERIA

- User can select Before Food or After Food.
- Selected condition is visually clear.
- Pathway state updates from returned data.
- Invalid or unavailable conditions are handled safely.



FILES / FOLDERS

src/components/pathway/FoodConditionSelector.tsx
src/services/pathwayService.ts
src/state/pathwayState.ts
src/types/pathway.ts



DELIVERABLES

- Food condition selector
- Variation handling
- Updated visualization



DEPENDENCIES

Task 05, Task 08.

ESTIMATED TIME

3–4 Hours

NOTES

Use the backend as the source of truth. ■ Do not implement pharmacological rules independently in the frontend.

■ Task 10 — Frontend Integration and Responsive Polish

Integrate completed frontend pieces and prepare the explorer for backend integration and team testing. Resolve UI inconsistencies and verify supported screen sizes.

CONCEPTS

- Component integration
- Responsive design
- Accessibility
- Error handling



ACCEPTANCE CRITERIA

- Explorer works as one coherent page.
- 3D visualization and information panel work together.
- Mobile layout remains usable.
- Loading and error states are consistent.



FILES / FOLDERS

src/

src/pages/

src/components/

src/styles/

DELIVERABLES

- Integrated frontend explorer
- Responsive layout
- Frontend quality baseline

DEPENDENCIES

Tasks 04–09.

ESTIMATED TIME

4–5 Hours

NOTES

Run the frontend against mock API responses before backend completion. ■ Do not wait until the final week to test integration.

■ Task 11 — Backend Application Bootstrap

Create the Node.js + TypeScript backend and establish the modular-monolith structure with clear module boundaries.

CONCEPTS

- Node.js
- TypeScript
- Modular monolith
- Configuration

ACCEPTANCE CRITERIA

- Backend starts successfully.
- TypeScript compilation succeeds.
- Environment configuration is supported.
- Module structure is established.

FILES / FOLDERS

```
backend/  
src/app.ts  
src/server.ts  
src/config/  
src/modules/  
package.json  
tsconfig.json  
.env.example
```

DELIVERABLES

- Working backend
- Modular structure
- Configuration system

DEPENDENCIES

Task 01.

ESTIMATED TIME

3–4 Hours

NOTES

Keep modules cohesive. ■ Do not create microservices for the MVP.

■ Task 12 — PostgreSQL Database and Migration Setup

Configure PostgreSQL and establish reproducible migrations so database changes are version-controlled.

CONCEPTS

- PostgreSQL
- Migrations
- Schema versioning
- Environment isolation

ACCEPTANCE CRITERIA

- PostgreSQL connection works.
- Migrations create the schema.
- Database configuration comes from environment variables.
- Schema changes are version-controlled.

FILES / FOLDERS

```
backend/src/config/database.ts
backend/src/database/
backend/src/database/migrations/
.env.example
```

DELIVERABLES

- PostgreSQL connection
- Migration system
- Database configuration

DEPENDENCIES

Task 11.

ESTIMATED TIME

3–4 Hours

NOTES

Use migrations for every schema change. ■ Avoid undocumented manual production changes.

■ Task 13 — Core Domain Database Schema

Implement the relational model for drugs, pathways, nodes, edges and content, supporting future drugs without drug-specific tables.

CONCEPTS

- Relational modelling
- Foreign keys
- Graph representation
- Normalization

ACCEPTANCE CRITERIA

- Drug records are supported.
- Pathways belong to drugs.
- Nodes and edges are relationally represented.
- Node content is separate from graph structure.

[illegible]

FILES / FOLDERS

```
backend/src/database/migrations/
backend/src/modules/drugs/
backend/src/modules/pathways/
backend/src/modules/content/
```

[illegible]

DELIVERABLES

- Core schema
- Foreign-key relationships
- Database constraints

[illegible]

DEPENDENCIES

Task 12.

ESTIMATED TIME

5–6 Hours

NOTES

- Do not create tables such as `ibuprofen` nodes.

■ Task 14 — Pathway Variation and Food Condition Model

Implement Before Food / After Food without duplicating the entire pathway. Variations modify applicable information while retaining one underlying pathway structure.

CONCEPTS

- Parameters
- Parameter values
- Variations
- Reusable pathway structure

[REDACTED]

ACCEPTANCE CRITERIA

- Food condition values are represented.
- Base pathway can have condition-specific variations.
- Full pathways are not duplicated.
- Invalid condition values are rejected.

[REDACTED]

FILES / FOLDERS

```
backend/src/database/migrations/
```

backend/src/modules/pathways/

backend/src/modules/pathways/models/

DELIVERABLES

- Variation schema
- Food condition model
- Data integrity rules

DEPENDENCIES

Task 13.

ESTIMATED TIME

4-5 Hours

NOTES

Keep the model extensible. ■ Do not invent additional MVP parameters.

■ Task 15 — Pathway REST APIs

Implement REST APIs for drugs, pathways, nodes, edges and condition-specific data.

CONCEPTS

- REST
- Controllers
- Routes
- Services
- DTOs
- API errors

ACCEPTANCE CRITERIA

- Pathway endpoints return agreed JSON structures.
- Invalid identifiers are handled safely.
- Missing resources return appropriate status codes.
- Response format matches the shared contract.

FILES / FOLDERS

```
backend/src/modules/pathways/controllers/
```

```
backend/src/modules/pathways/routes/
```

```
backend/src/modules/pathways/services/
```

backend/src/modules/pathways/dto/

DELIVERABLES

- Pathway endpoints
- Node/edge endpoints
- API response models

[illegible]

DEPENDENCIES

Tasks 13–14.

ESTIMATED TIME

5–6 Hours

NOTES

Use controllers for HTTP concerns and services for business logic. ■ Never trust client input.

■ Task 16 — Quiz and Progress Database/API

Implement quiz and progress backend functionality. Authenticated users can save relevant results while anonymous users can still take the quiz.

CONCEPTS

- Quiz modelling
- Attempts
- User progress
- Persistence
- Authorization

[REDACTED]

ACCEPTANCE CRITERIA

- Quiz questions can be retrieved.
- Quiz submissions are validated server-side.
- Authenticated results can be persisted.
- Anonymous participation does not require an account.

[illegible]

FILES / FOLDERS

```
backend/src/modules/quiz/
```

```
backend/src/modules/progress/
```

```
backend/src/database/migrations/
```

DELIVERABLES

- Quiz schema
- Quiz API
- Progress API

[illegible]

DEPENDENCIES

Task 13.

ESTIMATED TIME

5–6 Hours

NOTES

- Never trust answer correctness information supplied by the frontend.

■ Task 17 — Authentication and Google OAuth/OIDC

Implement optional Google authentication and establish the authenticated identity required for progress persistence.

CONCEPTS

- OAuth/OIDC
- Authentication
- Sessions/tokens
- Identity
- Secure configuration

ACCEPTANCE CRITERIA

- User can authenticate through the agreed Google flow.
- Authentication state is established securely.
- Invalid authentication attempts are rejected.
- OAuth secrets remain server-side.

FILES / FOLDERS

backend/src/modules/auth/
backend/src/modules/auth/controllers/
backend/src/modules/auth/services/
backend/src/middleware/
backend/src/config/

DELIVERABLES

- Google authentication
- User identity handling
- Authentication middleware

DEPENDENCIES

Task 11.

ESTIMATED TIME

5–7 Hours

NOTES

Use established OAuth/OIDC libraries. ■ Never expose Google client secrets.

■ Task 18 — Authorization and User Data Protection

Implement server-side authorization for user progress and administrative operations. Users must only access resources they are authorized to access.

CONCEPTS

- Authorization
- RBAC
- Ownership
- Least privilege
- Access control

ACCEPTANCE CRITERIA

- Authenticated users access their own progress.
- Users cannot access another user's private progress.
- Admin operations require admin authorization.
- Authorization is enforced server-side.

FILES / FOLDERS

backend/src/middleware/auth.ts
backend/src/middleware/authorization.ts
backend/src/modules/users/
backend/src/modules/admin/

DELIVERABLES

- Authorization middleware
- Ownership checks
- Admin access control

DEPENDENCIES

Task 17.

ESTIMATED TIME

4–5 Hours

NOTES

Authentication identifies the user; authorization determines allowed actions. ■ Never rely on frontend protection.

■ Task 19 — Validation, Error Handling and Security Middleware

Add centralized validation, safe errors and common API security controls. All external input is treated as untrusted.

CONCEPTS

- Input validation
- Error handling

- Parameterized queries
- Rate limiting
- Security headers
- CORS

ACCEPTANCE CRITERIA

- Request bodies and parameters are validated.
- Invalid requests return safe errors.
- Database queries use parameterized mechanisms.
- Security middleware is configured appropriately.

FILES / FOLDERS

backend/src/middleware/
 backend/src/validation/
 backend/src/errors/
 backend/src/config/security.ts

DELIVERABLES

- Validation layer
- Error-handling layer
- Security middleware

DEPENDENCIES

Tasks 15–18.

ESTIMATED TIME

5–6 Hours

NOTES

- Never return stack traces, secrets or internal database details.

■ Task 20 — Backend Integration and API Contract Freeze

Integrate backend modules and verify that API behavior matches the frontend contract. This creates the stable backend contract for full-team integration.

CONCEPTS

- API integration
- Contract verification
- Module integration
- API documentation

ACCEPTANCE CRITERIA

- All MVP endpoints work together.

- Response structures match the frontend contract.
- Authentication and authorization work on protected endpoints.
- API documentation reflects implementation.

FILES / FOLDERS

backend/src/
docs/api/
backend/package.json

DELIVERABLES

- Integrated backend
- Stable API contract
- API documentation

DEPENDENCIES

Tasks 13–19.

ESTIMATED TIME

5–6 Hours

NOTES

- Do not make breaking API changes after this point without team agreement.

■ Task 21 — Scientific Content Structure and Data Preparation

Prepare curated Ibuprofen educational content using the agreed pathway and content structure. Content should be understandable to general users while maintaining scientific accuracy.

CONCEPTS

- Educational content
- Content structure
- Scientific validation
- Node metadata

ACCEPTANCE CRITERIA

- Ibuprofen pathway content follows the approved structure.
- Required node information exists.
- Content matches the intended accessibility level.
- Scientific content is reviewed before production use.

FILES / FOLDERS

docs/content/
data/
data/pathway/
data/content/

DELIVERABLES

- Ibuprofen pathway content
- Node definitions
- Node roles and available facts

DEPENDENCIES

Tasks 13–14.

ESTIMATED TIME

5–7 Hours

NOTES

Keep scientific facts separate from UI wording. ■ Do not invent pharmacological claims.

■ Task 22 — Ibuprofen Food-Condition Content

Prepare validated content and pathway differences for Before Food and After Food, limited to approved MVP behavior.

CONCEPTS

- Parameterized content
- Pathway variation

- Scientific review
- Conditional content

ACCEPTANCE CRITERIA

- Before Food content is defined.
- After Food content is defined.
- Only validated differences are represented.
- Content maps correctly to the variation model.

FILES / FOLDERS

data/content/
data/pathway/variations/
docs/content/

DELIVERABLES

- Food-condition content
- Variation mapping
- Content validation record

DEPENDENCIES

Task 21, Task 14.

ESTIMATED TIME

3–4 Hours

NOTES

- Do not create unsupported visual differences merely because they look interesting.

■ Task 23 — Quiz Content and Question Bank

Create the MVP Ibuprofen quiz and its question/option structure. Questions should test understanding of the visualized pathway.

CONCEPTS

- Question modelling
- Multiple-choice questions
- Answer validation
- Educational assessment

ACCEPTANCE CRITERIA

- Quiz questions follow the agreed schema.
- Each question has valid options.
- Correct answers are defined server-side.

■ Questions are understandable to the intended audience.

FILES / FOLDERS

data/quiz/

docs/quiz/

DELIVERABLES

- Ibuprofen quiz
- Question bank
- Answer definitions

DEPENDENCIES

Task 16, Task 21.

ESTIMATED TIME

3–4 Hours

NOTES

Avoid ambiguous questions.

■ Task 24 — Content Loading and Database Seeding

Create reproducible seed data for the Ibuprofen pathway, content and quiz so every developer can work with the same dataset.

CONCEPTS

- Database seeding
- Reproducibility
- Test data
- Data integrity

ACCEPTANCE CRITERIA

- Seed process creates required MVP data.
- Seed process is repeatable according to the chosen strategy.
- Foreign-key relationships remain valid.
- Team members can reproduce the same local dataset.

FILES / FOLDERS

backend/src/database/seeds/

data/

backend/src/database/

DELIVERABLES

- Ibuprofen seed data

-

Tasks 13–16, 21–23.

4–5 Hours

Keep seed data deterministic. ■ Never use production credentials or sensitive personal data.

Implement the minimum administrative functionality required to maintain pathway, content and quiz information with proper authorization.

- Admin workflows
- CRUD
- Authorization
- Content validation
- Auditability

- Authorized administrators can manage approved content.
- Unauthorized users cannot access admin operations.
- Content input is validated server-side.
- Invalid pathway relationships cannot be saved.

```
backend/src/modules/admin/  
backend/src/modules/content/  
src/pages/admin/  
src/components/admin/
```

- Admin content workflow
- Admin UI
- Content validation

Tasks 18–24.

ESTIMATED TIME

5–7 Hours

NOTES

Keep admin functionality minimal. ■ Do not expose database-level operations directly to the browser.

■ Task 26 — Security Test Suite

Create security-focused tests for authentication, authorization, validation, input handling and user-data isolation.

CONCEPTS

- OWASP risks
- Authorization testing
- Input validation
- Security regression testing
- Negative testing

ACCEPTANCE CRITERIA

- Unauthorized requests are rejected.
- Cross-user progress access is rejected.
- Admin endpoints reject normal users.
- Malformed and malicious inputs are safely handled.

FILES / FOLDERS

tests/security/
tests/api/
tests/fixtures/

DELIVERABLES

- Security test cases
- Negative test suite
- Authorization test suite

DEPENDENCIES

Tasks 18–20, 25.

ESTIMATED TIME

5–6 Hours

NOTES

- Passing functional tests does not mean the application is secure.

■ Task 27 — Full API and Integration Testing

Test communication between frontend-facing APIs, backend modules and PostgreSQL using real application data.

CONCEPTS

- API testing
- Integration testing
- Database integration
- Contract testing
- Error scenarios

ACCEPTANCE CRITERIA

- Pathway APIs work against real database data.
- Food-condition changes return correct data.
- Quiz submission works end-to-end through the API.
- API/database failure cases are handled safely.

FILES / FOLDERS

tests/integration/
tests/api/
tests/fixtures/

DELIVERABLES

- API integration tests
- Database integration tests
- Failure-case tests

DEPENDENCIES

Tasks 20, 24.

ESTIMATED TIME

5–6 Hours

NOTES

Use isolated test data where possible.

■ Task 28 — Frontend + Backend Full Integration

Connect the production frontend to the completed backend and replace mock data with real API data. This is the primary team integration task.

CONCEPTS

- Full-stack integration
- API contracts
- State management
- Authentication integration
- Error handling



ACCEPTANCE CRITERIA

- Frontend retrieves the real Ibuprofen pathway.
- Node information comes from backend data.
- Before Food / After Food works through the real API.
- Quiz submission and progress persistence work.



FILES / FOLDERS

src/
backend/src/
tests/e2e/
docs/integration/



DELIVERABLES

- Fully integrated application
- Real API data flow
- Integrated authentication/progress



DEPENDENCIES

Tasks 10, 20, 24–27.

ESTIMATED TIME

6–8 Hours

NOTES

This should be performed by all three members together. ■ Do not make this a final-day activity.

■ Task 29 — End-to-End Testing and MVP Acceptance

Execute complete MVP use cases from real user and administrator perspectives and confirm the implementation satisfies agreed requirements.

CONCEPTS

- E2E testing
- Use-case testing
- Acceptance testing
- Regression testing
- Requirement traceability



ACCEPTANCE CRITERIA

- Anonymous user can explore Ibuprofen.
- User can inspect nodes.
- User can switch Before Food / After Food.
- User can complete quiz.

- Google authentication and progress saving work.
- Admin content workflow works.

FILES / FOLDERS

tests/e2e/

tests/use-cases/

docs/testing/

docs/traceability/

DELIVERABLES

- E2E test results
- Use-case acceptance results
- Requirement traceability
- MVP defect list

DEPENDENCIES

Task 28.

ESTIMATED TIME

6–8 Hours

NOTES

- Do not mark MVP complete while critical security, data-integrity or core-functionality defects remain.

Prepare the application for deployment and perform the final release review. Confirm that the system is operationally ready, not merely functional on developer machines.

CONCEPTS

- Production deployment
- Environment configuration
- Monitoring
- Backup and recovery
- Release management

ACCEPTANCE CRITERIA

- Production configuration is separated from development.
- Secrets are stored securely outside source control.
- Backup/recovery strategy is defined.
- Logging and basic monitoring are available.
- Final MVP acceptance is approved.

FILES / FOLDERS

docs/deployment/
docs/operations/
docs/release/
.env.example
README.md

DELIVERABLES

- Production deployment configuration
- Release checklist
- Backup/recovery procedure
- Operations documentation
- MVP release

DEPENDENCIES

Task 29.

ESTIMATED TIME

5–7 Hours

NOTES

- Never deploy using development secrets, debug configuration or unrestricted administrative access.

Final Team Execution Model

Member 1: Tasks 01–10 — Frontend + 3D

Member 2: Tasks 11–20 — Backend + Database + APIs

Member 3: Tasks 21–30 — Content + Quiz + QA/Security + Integration

All three members share API contracts, data contracts, integration checkpoints and final acceptance.

Critical Integration Checkpoints

Checkpoint	Members	Expected Result
CP-01	1 + 2	Frontend successfully consumes one real API response
CP-02	1 + 2 + 3	Real Ibuprofen pathway appears in the 3D explorer
CP-03	1 + 2 + 3	Before Food / After Food works correctly
CP-04	1 + 2 + 3	Quiz submission and result flow works
CP-05	1 + 2	Google authentication and saved progress work
CP-06	All	Admin/content workflow works
CP-07	All	Security and E2E tests pass
CP-08	All	MVP release approved

Non-Negotiable Team Rules

- One API contract.
- One agreed database schema.
- No direct pushes to main.
- No secrets in Git.
- No scientific logic duplicated independently across frontend/backend.
- No frontend-only security decisions.
- No drug-specific code paths when data-driven implementation is possible.
- Integrate continuously — never wait until Task 28 to discover compatibility problems.
- Every completed task must satisfy its acceptance criteria before dependent work is treated as complete.
- Any breaking architecture/API/database change must be discussed by all three members before implementation.